

Non-Graded Project: Ship-Ready LLM Assistant: From Prompt to Production

Description

In this project, you will build a small, but production-minded LLM assistant that brings together everything you've learned in the course. You'll design a structured prompt that avoids silent failures, wrap the OpenAI API in a resilient middleware service, and connect it to a React frontend that streams responses and captures real user feedback. By the end, you'll have a working prototype that doesn't just “call a model,” but behaves like a trustworthy, scalable product.

Learning Objectives

By the end of this project, learners will be able to:

- Design a structured LLM prompt with clear roles, constraints, and output formatting that reduces ambiguity and hallucinations.
- Implement a backend middleware service that adds retries, caching, and token usage logging around LLM API calls.



- Build a minimal React-based UI that streams model responses and captures user feedback (ratings, flags, clarifications).
- Integrate prompt, backend, and UI into a cohesive system that behaves reliably under realistic usage.
- Reflect on trade-offs and design decisions that affect reliability, cost, and user trust in LLM applications.



Project Components

Introductory Slide

Title: Launch Day: Your LLM Assistant Has to Work

Subtitle: Design the prompt, backend, and UI that turn a raw model into a reliable product

Tools to be used:

- OpenAI API
- FastAPI (Python) or Node.js (Express/Nest) for middleware
- React + Vercel AI SDK for the frontend
- Postman (or similar) for API testing
- Optional: In-memory cache (e.g., simple Python dict, Node Map, or Redis if available)

Scenario

Your client is an early-stage AI startup building a customer-facing assistant to answer product and policy questions. They've had a quick prototype in place that calls the OpenAI API directly from the frontend with a single, loosely written prompt.

- It looks impressive in simple demos—but under real usage it:
- Occasionally hallucinates policy details and presents them confidently
- Times out or fails during traffic spikes
- Shows raw error messages to users

- Has no way to collect feedback when answers are unhelpful or confusing

The startup is preparing for a high-stakes demo with potential investors who expect to see **more than just a chat box**. They want to see:

- A prompt that keeps the assistant within clear rails
- A backend that can handle failures, rate limits, and token costs
- A UI that feels responsive, transparent, and capable of learning from users

You've been hired to deliver a **single prototype** that showcases this “from prompt to production” mindset. You don't have to build a massive system; you do have to show that your assistant is **optimized, resilient, and feedback aware**.

Instructions for Learners

Step 1: Define the Assistant and Its Prompt

Choose a simple but realistic assistant use case (e.g., refund policy helper, FAQ assistant, onboarding guide).

Draft a **structured prompt** that includes:

- A clear role (e.g., “You are a support assistant for...”).
- A precise task description.
- Constraints (e.g., “Do not guess,” “Refer only to the policy block below”).
- A small JSON schema or clearly defined sections for the output.

Add reasoning guidance: step-by-step thinking or constrained reasoning (e.g., “Think internally step-by-step, then respond with a short final explanation”).

Test your prompt in the OpenAI Playground or via a simple script to confirm it produces stable, structured outputs.

Save your final prompt in a dedicated file (e.g., prompt_spec.md or prompt.json).

Outcome: A refined, production-ready prompt that minimizes ambiguity and defines a predictable response shape.

Step 2: Build the Middleware Service

Set up a small backend using FastAPI (Python) or Node.js:

- Create an endpoint (e.g., POST /api/chat) that accepts user messages.
- Wrap the OpenAI API call in a retry-safe, timeout-aware function.

Implement:

- **Timeouts:** Ensure calls do not hang indefinitely.
- **Retries with exponential backoff:** Retry transient failures (e.g., 429, network hiccups) without spamming.
- **Caching:** Cache repeated prompts + responses (even simple in-memory caching is fine).
- **Token logging:** Log estimated tokens per request and response (or OpenAI usage where available).

Use **Postman** (or similar) to:

- Send test requests to your endpoint.
- Observe responses, error behavior, and log output.

Outcome: A resilient backend endpoint that shields the frontend from raw model failures and exposes a clean, predictable interface.

Step 3: Build the Minimal React UI

Create a simple React interface using the Vercel AI SDK (or a custom fetch hook):

- Input box for the user's question.
- Display area for the assistant's responses.
- Show messages in a clear chat layout (user vs. assistant).

Enable **streaming responses** if possible, so users see the answer appear progressively.

Ensure that the frontend communicates only with your **backend endpoint**, not directly with OpenAI.

Outcome: A responsive chat UI that uses your backend as the single gateway to the LLM.

Step 4: Integrate Feedback Capture

For each assistant response, add quick feedback controls:

- Thumbs up / thumbs down
- A “Flag” button for problematic answers
- A short text field for clarifications when feedback is negative (e.g., “What didn’t work?”).

Send feedback data to the backend as structured JSON, including:

- The message ID or timestamp
- Rating (up/down or numeric)
- Any user notes / clarifications

In your backend, log or store this feedback (e.g., in-memory list, simple file, or placeholder for a database).

Optionally, display a **basic metric** in the UI (e.g., count of thumbs up vs. thumbs down in this session).

Outcome: An end-to-end loop where every response can be rated, flagged, and analyzed later for improvement.

Step 5: End-to-End Test and Reflection

Run several full conversations through your assistant:

- Try edge cases (ambiguous questions, incomplete information).
- Intentionally cause transient errors (e.g., by tweaking your timeout) if feasible to see retry behavior.
- Trigger repeated questions to confirm caching is working.

Observe:

- Does the prompt keep the assistant inside the correct policy or domain?
- Does the backend handle failures gracefully without exposing raw errors?
- Does the UI feel responsive and transparent?
- Is feedback capture easy and natural for the user?

Write a **~300-word reflection** explaining:

- How your prompt design reduced silent failures.
- How your middleware improved reliability and token visibility.
- How your UI and feedback loop affected user trust and perceived quality.

Outcome: A thoughtful reflection that ties together LO1 (prompt), LO2 (backend), and LO3 (UX & feedback).

Compile Your Findings

Requirement 1 — Optimized Prompt Specification

Include:

- Final optimized prompt specification.
- Clearly defined role instructions.
- All constraints and behavioral boundaries.
- Output format requirements (JSON schema or structured sections).
- File upload of the completed prompt specification.

Requirement 2 — Middleware Service (Backend Code)

Include:

- Backend middleware wrapping the OpenAI API.
- Retry logic, timeout handling, and caching strategy.
- Token-usage logging.
- Any helper or configuration files needed to understand the backend.
- Optional short video demonstrating endpoint behavior.

Requirement 3 — Frontend UI with Feedback Integration

Include:

- React UI implementation.
- Demonstration of user message submission and model response.
- Feedback mechanisms (rating, flag, clarification, etc.).
- Optional short video showing the end-to-end UI workflow.
- Optional UI screenshots.

Requirement 4 — Integration & Reflection Document

Include:

- A 300-word reflection detailing how the prompt, backend, and frontend integrate.
- Explanation of how the system improves reliability, scalability, and user trust.
- Key design decisions you are most proud of.
- Improvements you would make with additional time.
- Optional audio or video reflection.

Final Outcome

By the end of this project, learners deliver a small but production-ready LLM assistant that combines a structured, ambiguity-free prompt, a resilient backend with retries, timeouts, caching, and token logging, and a minimal React UI that streams responses and captures user feedback. The frontend talks only to the middleware, ensuring reliable behavior even under failures, while the feedback loop (ratings, flags, clarifications) demonstrates how real products learn from users. Learners submit four items—a refined prompt, backend code, frontend implementation, and a short reflection—showing they can integrate all layers into a trustworthy end-to-end LLM application.

Summary

This course end project guides you through building a production-ready LLM assistant by designing a structured prompt, creating a robust backend with retries and caching, and developing a simple React UI that streams responses and captures user feedback. Set in a realistic startup scenario, it requires transforming a fragile prototype into a reliable, investor-ready tool. You submit the final prompt, backend, frontend, and a brief reflection explaining how your integrated system improves reliability, scalability, and user trust.

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.